

23CS312	DESIGN AND ANALYSIS OF ALGORITHMS	L	T	P	C
		3	0	2	4
COURSE OBJECTIVES:					
- To understand and apply the algorithm analysis techniques on searching and sorting algorithms - To critically analyze the efficiency of graph algorithms - To understand different algorithm design techniques - To solve programming problems using state space tree - To understand the concepts behind NP Completeness, Approximation algorithms and randomized algorithms					
UNIT I	INTRODUCTION				9
Time and space complexity - Asymptotic Notations - Solving Recurrences: substitution method - Lower bounds - hash function - searching: linear search, binary search and Interpolation Search, String Matching: The naïve string - matching algorithm - Rabin-Karp algorithm - Sorting: Insertion sort, heap sort					
UNIT II	GRAPH ALGORITHMS				9
Representations of graphs - Graph traversal: DFS - BFS - Minimum spanning tree: Kruskal's and Prim's algorithm - Shortest path: Bellman - Ford algorithm - Dijkstra's algorithm - Maximum flow: Flow networks - Ford-Fulkerson method - Maximum bipartite matching.					
UNIT III	ADVANCED DESIGN AND ANALYSIS TECHNIQUES				9
Divide and Conquer methodology: Merge sort - Quick sort- Dynamic programming: Elements of dynamic programming - Matrix-chain multiplication - Multi stage graphs. Greedy Technique: Elements of the greedy strategy - Activity-selection problem - Huffman Trees					
UNIT IV	STATE SPACE SEARCH ALGORITHMS				9
Backtracking: n-Queens problem - Hamiltonian Circuit Problem - Subset Sum Problem - Graph colouring problem Branch and					

Bound : Solving 15-Puzzle problem - Assignment problem - Knapsack Problem - Travelling Salesman Problem.		
UNIT V	NP-COMPLETE AND APPROXIMATION ALGORITHM	9
Tractable and intractable problems: Polynomial time algorithms - Venn diagram representation - Non Deterministic algorithms - NP-hardness and NP-completeness - Problem reduction: TSP - 3 CNF problem. Approximation Algorithms: Bin Packing problem - Randomized Algorithms: concept and application - primality testing - randomized quick sort.		
TOTAL: 45 PERIODS		
PRACTICAL EXERCISES: 30 PERIODS		
- Sort a given set of elements using the Quick sort method and determine the time required to sort the elements. Repeat the experiment for different values of n, the number of elements in the list to be sorted. The elements can be read from a file or can be generated using the random number generator. - Implement a Merge Sort algorithm to sort a given set of elements and determine the time required to sort the elements. Repeat the experiment for different values of n, the number of elements in the list to be sorted. The elements can be read from a file or can be generated using the random number generator. (A) Obtain the Topological ordering of vertices in a given digraph. (B) Compute the transitive closure of a given directed graph using Warshall's algorithm. - Implement 0/1 Knapsack problem using Dynamic Programming. - From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm - Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.		

7. (A) Print all the nodes reachable from a given starting node in a digraph using BFS method. (B) Check whether a given graph is connected or not using DFS method. 8. Find a subset of a given set $S = \{s_1, s_2, \dots, s_N\}$ of n positive integers whose sum is equal to a given positive integer d. For example, if $S = \{1, 2, 5, 6, 8\}$ and $d = 9$ there are two solutions $\{1, 2, 6\}$ and $\{1, 8\}$. A suitable message is to be displayed if the given problem instance doesn't have a solution. 9. Implement any scheme to find the optimal solution for the Traveling Salesperson problem and then solve the same problem instance using any approximation algorithm and determine the error in the approximation. 10. Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. 11. Implement All-Pairs Shortest Paths Problem using Floyd's algorithm. 12. Implement N Queen's problem using Back Tracking.	
COURSE OUTCOMES:	
After completion of the course, the students will be able to:	
CO1:	Apply recursive and non-recursive algorithms to solve problem.
CO2:	Apply appropriate framework to meet algorithm's efficiency.
CO3:	Apply graph algorithms to solve problems and analyze their efficiency.
CO4:	Solve problems using algorithm design techniques like divide and conquer, dynamic programming and greedy techniques.
CO5:	Apply State Space Tree Analysis for Problem-Solving.
CO6:	Solve problems using approximation algorithms and randomized algorithms

TEXT BOOKS:																
1	Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein, “Introduction to Algorithms”, 3rd Edition, Prentice Hall of India, 2009.															
2	Ellis Horowitz, SartajSahni, SanguthevarRajasekaran Computer Algorithms/C++ Orient Blackswan, 2nd Edition, 2019.															
REFERENCES:																
1	Anany Levitin, “Introduction to the Design and Analysis of Algorithms”, 3rd Edition, Pearson Education, 2012.															
2	Alfred V. Aho, John E. Hopcroft and Jeffrey D. Ullman, "Data Structures and Algorithms", Reprint Edition, Pearson Education, 2006.															
3	S. Sridhar, “Design and Analysis of Algorithms”, Oxford university press, 2014.															
COs		POs												PSOs		
		1	2	3	4	5	6	7	8	9	10	11	12	1	2	3
1	3	2	1	1	2	-	-	-	-	-	-	-	3	2	-	
2	3	2	1	1	2	-	-	-	-	-	-	1	3	2	-	
3	3	2	1	1	2	-	-	-	-	-	-	-	3	2	-	
4	3	2	1	1	2	-	-	-	-	-	-	1	3	2	-	
5	3	2	1	1	2	-	-	-	-	-	-	-	3	2	-	
6	3	2	1	1	2	-	-	-	-	-	-	-	3	2	-	
Overall Correlation	3	2	1	1	2	-	-	-	-	-	-	1	3	2	-	
Recommended by Board of Studies										08-04-2024						
Approved							2 nd ACM			Date			25-05-2024			